

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

- Performance Measure (P) – Measures how well the agent is doing (e.g., score or rewards).
- Environment (E) – The world where the agent operates (e.g., grid, walls, food).
- Actuators (A) – The actions the agent can perform to interact with the environment.
- Sensors (S) – The information the agent receives from the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

- self.width and self.height – define the size of the grid.
- self.walls – represents obstacles in the environment.
- self.food_positions – stores the locations of food items.
- self.opponents – represents other agents in the environment.
- self.agent_pos – represents the agent's current position in the grid.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment is Multi-Agent because it contains `self.opponents`, which represent other agents that can move and interact with the main agent. The environment includes more than one agent, so their actions can affect the overall outcome of the simulation.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the information an agent has received from its sensors over time. It helps the agent understand the environment and decide what action to take.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) component of the PEAS framework. It collects information from the environment, such as the agent's position, food, walls, and toxic traps, which the agent uses to make decisions.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what

the agent can and cannot see.

The environment is Partially Observable because the agent only receives limited information from `get_percept()`, such as its position, nearby conditions, food detection, walls, and toxic traps. It cannot see the complete grid state, including the locations of all food, opponents, and obstacles, so it does not have full knowledge of the environment.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The PEAS component being updated is the Performance Measure (P). When the agent hits a wall, `execute_action()` reduces the score, which is used to measure how well the agent is performing in the environment.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the performance measure should evaluate the agent's effect on the environment, not how much internal processing it does. In this case, losing points for hitting a wall shows whether the agent is making good decisions while interacting with the environment. This helps measure the agent's actual success.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable because the agent cannot detect the hidden `self.toxic_traps` information through its sensors. The traps exist in the environment, but since they are not included in `get_percept()`, the agent cannot know their locations and may make unsafe decisions.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the `smells_toxin` sensor improves the agent's rationality because it gives the agent more information about the environment. The agent can detect dangerous locations and avoid toxic traps, helping it make safer decisions and maximize its expected score.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score`

by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World Exploit was when the agent was rewarded only for cleaning dirt, so it could exploit the metric by repeatedly cleaning an already clean location to increase its score without actually improving the environment. This showed that a poorly designed performance measure can encourage unwanted

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING